Universidad del Valle de Guatemala

CC2008 - Introducción a la Programación Orientada a Objetos

Semestre II, 2026



Laboratorio 3

Herencia

Josue David Garcia Mendez 26375

1. Requisitos funcionales

1.1 Información que el sistema debe conservar

Grupo	Datos
Orden de servicio (común)	Código único, nombre del cliente, dirección, horas estimadas (decimal), zona (metropolitana o periférica), estado, número de visitas realizadas, monto cobrado al registrar, cargos acumulados por visitas adicionales y jornada a la que está asignada (si tiene)
Datos propios de cada categoría	Instalación: cantidad de puntos (entero positivo). Instalación eléctrica certificada: los de instalación más el valor declarado de materiales. Aire acondicionado: capacidad en BTU
Cuadrilla	Código único, capacidad máxima de horas por jornada, indicador de certificación en refrigeración
Jornada	Número de jornada, cuadrilla responsable, lista de órdenes, estado y resultados de las visitas registrados en esa jornada.
Ingresos	Total por registro, total por visitas adicionales e ingreso por categoría

1.2 Operaciones que el sistema debe permitir

1. Cotizar un servicio (sin registrar nada).
2. Registrar una orden de cualquier categoría.
3. Registrar una cuadrilla.
4. Crear una jornada para una cuadrilla.
5. Asignar una orden pendiente a una jornada.
6. Retirar una orden de una jornada en preparación.
7. Iniciar una jornada.
8. Registrar el resultado de la visita de cada orden.
9. Consultar órdenes y cuadrillas.
10. Generar los tres reportes de gerencia.

1.3 Condiciones de validez por operación

Operación	Condiciones para que sea válida	Si no se cumple
Cotizar	Los mismos datos y validaciones que registrar la orden (el código no se exige porque no se registra nada)	Mensaje de error. No se registra nada
Registrar orden (común)	Código no vacío y no repetido; cliente y dirección no vacíos; horas estimadas > 0; zona válida	Se rechaza con mensaje. No se incorpora la orden al catálogo ni se genera un cobro

Diagnóstico	Horas estimadas ≤ 1.5	Se rechaza
Instalación	Puntos: entero positivo	Se rechaza
Instalación eléctrica certificada	El valor declarado de los materiales debe ser estrictamente mayor a cero (únicamente entradas numéricas positivas). Esta validación se realiza de forma previa al registro de la orden o a la generación de cualquier cobro.	Se rechaza
Mantenimiento de A/C	Zona metropolitana; BTU entre 9,000 y 60,000	Se rechaza
Registrar cuadrilla	Código no vacío y no repetido; capacidad > 0	Se rechaza
Crear jornada	La cuadrilla existe; no tiene otra jornada sin cerrar	Se rechaza
Asignar orden	La orden existe y está pendiente (no asignada a otra jornada); la jornada está en preparación; la suma de horas estimadas con la nueva orden no supera la capacidad; la orden es compatible con la cuadrilla	Se rechaza. La jornada queda igual
Retirar orden	La jornada está en preparación; la orden pertenece a esa jornada	Se rechaza
Iniciar jornada	La jornada está en preparación y tiene al menos una orden	Se rechaza
Registrar resultado	La jornada está en atención; la orden pertenece a ella y aún no tiene resultado	Se rechaza
Consultar / reportes	No requieren condiciones y no modifican datos	No aplica

Regla general: toda operación inválida se va a rechazar con un mensaje claro y no modifica el estado previo ni los ingresos. Y si hubiera una entrada con formato incorrecto (por ejemplo, texto donde se espera un número) no termina el programa.

1.4 Reglas de cobro

Diagnóstico. Tarifa fija según la zona, sin importar las horas.

- Metropolitana: Q150. Periférica: Q225.

Instalación.

- Horas por puntos = puntos × 0.75
- Horas facturables = máx(horas estimadas, horas por puntos), redondeadas hacia arriba al entero (hora o fracción)
- Costo = tarifa base de zona (Q100 metropolitana, Q175 periférica) + Q90 × horas facturables

- Ejemplo: 2.5 h estimadas, 5 puntos, metropolitana → horas por puntos 3.75 → se facturan 4 h → 100 + 360 = Q460.
- Ejemplo con fracción: 2.1 h estimadas, 1 punto, metropolitana → máx(2.1, 0.75) = 2.1 → se facturan 3 h → 100 + 270 = Q370.
- Ejemplo periférica: 2 h estimadas, 2 puntos → máx(2, 1.5) = 2 h → 175 + 180 = Q355.

Instalación eléctrica certificada. Costo de instalación + Q120 de certificado + 5 % del valor declarado de materiales.

- Ejemplo: 460 + 120 + (0.05 × 2,000) = **Q680**.

Mantenimiento de aire acondicionado. Tarifa base de zona + Q110 × horas estimadas + Q80 de gas refrigerante.

- Ejemplo: 2 h, metropolitana → 100 + 220 + 80 = **Q400**.

Cargo por visita adicional. Q40 por cada visita después de la primera, igual para todas las categorías.

1.5 Tiempos de atención y política de visitas

Categoría	Días hábiles (metropolitana)	Días hábiles (periférica)	Visitas máximas
Diagnóstico	1	2	3
Instalación	3	5	3
Instalación eléctrica certificada	5 (3 + 2)	7 (5 + 2)	2
Mantenimiento de A/C	2	No aplica (solo metropolitana)	1

1.6 Reglas de visitas, cargos e ingresos

- Al registrar la orden se cobra el monto completo, y ese cobro cuenta como ingreso por registro.
- La visita se cuenta cuando la jornada inicia. Si es la segunda o posterior, se registra en ese momento un cargo de Q40 como ingreso por visitas adicionales. Asignar una orden no genera cargo.
- Si el resultado es *visita fallida* y la orden aún tiene visitas disponibles, vuelve a pendiente. Si ya agotó su máximo, queda cancelada y no puede asignarse de nuevo.
- Una jornada se cierra cuando todas sus órdenes tienen resultado registrado, y en ese momento la cuadrilla queda libre.
- No alteran los ingresos: cotizar, retirar órdenes de una jornada y cualquier operación rechazada.

1.7 Reportes

Reporte	Contenido

Órdenes por categoría y estado	Cantidad de órdenes de cada categoría desglosada por cada estado
Ingresos	Total general, separado en ingresos por registro e ingresos por visitas adicionales, y el ingreso por categoría
Jornadas no cerradas	Para cada una: cuadrilla, horas asignadas (estimadas) y capacidad disponible (capacidad − horas asignadas)

1.8 Alcance general y datos de precarga

El sistema será una aplicación de consola desarrollada en Java. Su menú permanecerá disponible hasta que el usuario seleccione la opción de salir. Los datos se conservarán únicamente en memoria durante la ejecución; no se utilizarán archivos de datos, bases de datos ni interfaz gráfica. Todos los montos monetarios se mostrarán con dos decimales.

Al iniciar, el sistema contará con al menos dos órdenes de cada una de las cuatro categorías, para un mínimo de ocho órdenes. Todas estarán pendientes, tendrán cero visitas realizadas y no tendrán ingresos por registro ni cargos por visitas adicionales. Aunque cada orden tendrá los datos necesarios para calcular su costo, su incorporación como dato inicial no representará un cobro.

También se cargarán al menos tres cuadrillas, con códigos únicos y capacidades positivas. Al menos una estará certificada en refrigeración y al menos una no tendrá esa certificación. Los datos iniciales permitirán observar las reglas de cobro e incluirán una instalación cuyas horas por puntos superen sus horas estimadas.

La precarga constituye una excepción a la operación normal de registro: las órdenes iniciales no generan ingresos, mientras que una orden registrada durante el uso del programa genera el cobro completo de su servicio. Las órdenes precargadas sí podrán generar cargos por visitas adicionales cuando corresponda.

Para el mantenimiento de aire acondicionado, la tarifa base corresponde a la tarifa estándar de zona (Q100 para la zona metropolitana y Q175 para la periférica). Dado que esta categoría solo se admite en zona metropolitana, su cálculo utiliza fijamente Q100.

2. Decisiones que dependen de la categoría del servicio

Introducción

Cada categoría de servicio tiene reglas propias de costo, tiempo, visitas, restricciones de cuadrilla y validación. El criterio para asignar cada decisión es: la toma la clase que posee la información necesaria para tomarla. Así, Jornada, el sistema coordinador y los reportes nunca necesitan conocer el tipo concreto de una orden: solo invocan métodos de OrdenServicio, y cada subclase responde a su manera (polimorfismo).

2.1 Decisiones que SÍ dependen de la categoría

#	Decisión	Clase que la toma	Cómo	Por qué
1	Costo del servicio	Cada subclase concreta	Método abstracto calcularCosto() en OrdenServicio	InstalacionElectricaCertificada reutiliza el cálculo de Instalacion con

				super.calcularCosto() y suma sus extras
2	Días hábiles de atención	Cada subclase	Método abstracto getDiasAtencion()	Cada categoría tiene su propia regla. La eléctrica se define como super.getDiasAtencion() + 2
3	Máximo de visitas permitidas	Cada subclase	Método abstracto getMaximoVisitas()	Es una política por categoría (3, 3, 2 y 1)
4	Compatibilidad con la cuadrilla	OrdenServicio (comportamiento por defecto) y MantenimientoAireAcondicionado (sobrescribe)	Método esCompatibleCon(Cuadrilla): la base devuelve true y el A/C exige certificación en refrigeración	La restricción nace del servicio y no de la jornada. Jornada solo pregunta y nunca verifica tipos
5	Validación de datos propios	Constructor de cada subclase	Lanzar excepción si el dato es inválido	Cada clase conoce sus límites: diagnóstico ≤ 1.5 h, puntos entero positivo, materiales válidos, BTU entre 9,000 y 60,000, A/C solo en zona metropolitana
6	Nombre de la categoría	Cada subclase	Método getCategoria()	Los reportes agrupan por categoría. Instalación y eléctrica certificada son categorías distintas aunque una herede de la otra
7	Datos propios que se conservan	Cada subclase	Atributos privados	Los puntos, materiales y BTU no tienen sentido en las demás categorías

8	Tarifa base de Diagnóstico (150/225)	Diagnostico	Lógica interna del cálculo de costo	Es una regla exclusiva de esa categoría y cambia por razones distintas a la tarifa base compartida

2.2 Decisión compartida entre categorías: tarifa base por zona

Aspecto	Resolución
Dato	Tarifa base por zona: Q100 metropolitana, Q175 periférica
Quién la usa	Instalacion (y por herencia InstalacionElectricaCertificada) y MantenimientoAireAcondicionado
Dónde vive	Clase Tarifas (final, constructor privado), con el método estático baseServicio(Zona)
Por qué	Es una regla de la empresa, no de un tipo de servicio. Las dos categorías necesitan el mismo valor pero no son del mismo tipo, así que se centraliza en un único punto de cambio y se evita duplicar el dato y el if de zona
Qué no incluye	La tarifa de diagnóstico, que permanece en Diagnostico: coincide en forma (depende de la zona) pero no en el dato, y no tiene por qué cambiar junto con la tarifa base

Principio de diseño aplicado: Cohesión y Separación de Responsabilidades (SRP). Los elementos con motivos de cambio comunes se agrupan en la misma clase, mientras que las reglas independientes se encapsulan por separado.

2.3 Decisiones que NO dependen de la categoría

#	Decisión	Clase responsable	Por qué
1	Código no vacío, horas > 0, zona definida, cliente y dirección no vacíos	Constructor de OrdenServicio	Son datos comunes a todas las órdenes, se validan una sola vez
2	Código de orden único	SistemaTecniHogar (coordinadora)	Una orden no posee visibilidad del resto del catálogo; únicamente la clase coordinadora con la colección global puede validar la unicidad.
3	Código de cuadrilla único	SistemaTecniHogar	Análogo al código de orden; la clase coordinadora valida la unicidad sobre el catálogo de cuadrillas

4	Cargo de Q40 desde la segunda visita	OrdenServicio	Es igual para todas las categorías. Lo que varía es solo el máximo de visitas
5	Cancelar al agotar visitas	OrdenServicio, consultando getMaximoVisitas()	La decisión es común, pero el dato viene de la subclase
6	Cambios de estado de la orden	OrdenServicio	Protege su propio estado y rechaza transiciones inválidas
7	Control de capacidad	Jornada	Suma horas **estimadas** (no facturables) y compara con la capacidad de la cuadrilla
8	Una sola jornada sin cerrar por cuadrilla	SistemaTecniHogar	Es una regla sobre cuadrillas y jornadas, no sobre órdenes
9	Acumular ingresos	OrdenServicio y SistemaTecniHogar	Cada orden conserva su monto de registro y sus cargos por visitas adicionales. SistemaTecniHogar coordina las operaciones y calcula los totales sumando los valores de las órdenes registradas.
10	Formato correcto de las entradas	Clase de interfaz de usuario	Es una responsabilidad de interfaz con el usuario, no del dominio

2.4 Conclusión del diseño de responsabilidades

Las decisiones de costo, días de atención, máximo de visitas, nombre de categoría y compatibilidad con la cuadrilla se consultan mediante los métodos definidos en OrdenServicio. Cada subclase proporciona el comportamiento que le corresponde. Jornada, SistemaTecniHogar y los reportes trabajan con referencias de OrdenServicio, sin comprobar la clase concreta de cada orden.

SistemaTecniHogar coordina las operaciones que pueden generar ingresos, pero cada orden conserva su monto cobrado por registro y sus cargos por visitas adicionales. Los reportes calculan los totales a partir de esos valores, sin mantener acumulados independientes que puedan contradecirlos.

Este diseño permite incorporar nuevas categorías sin cambiar las clases consumidoras mientras las nuevas reglas puedan expresarse mediante los métodos y datos existentes. Si una categoría exige información que todavía no está representada, como otra certificación de cuadrilla, será necesario ampliar la clase responsable de conservar esa información.

3. Estados y transiciones

Opción	Ventaja	Desventaja
Estado ASIGNADA	Un solo campo estado describe todo; la condición “no está asignada a otra jornada” se reduce a estado == PENDIENTE; los reportes por estado muestran cuántas órdenes están asignadas	Hay que mantener sincronizados el estado y la lista de órdenes de la jornada
Referencia a la jornada	No hay un estado más	La orden tendría dos datos para saber en qué situación está (estado y referencia), con riesgo de que se contradigan

Se modela ASIGNADA como un estado explícito para distinguir las órdenes reservadas en una jornada de las disponibles (PENDIENTE), permitiendo validar la disponibilidad con un único atributo y reflejar con precisión el estado en los reportes gerenciales.

3.1 Estados de la orden

Estado	Significado
Pendiente	Registrada y disponible para asignarse a una jornada
Asignada	Pertenece a una jornada que está en preparación
En atención	Su jornada ya inició y se espera el resultado de la visita
Completada	El servicio se realizó. Estado final
Cancelada	La última visita permitida terminó con resultado FALLIDA. Es un estado final y la orden no puede asignarse nuevamente.

3.2 Transiciones de la Jornada

Una jornada tiene tres estados: EN_PREPARACION, EN_ATENCION y CERRADA. En preparación se pueden asignar y retirar órdenes. En atención se registran los resultados de las visitas. Una jornada cerrada conserva su información para consulta y no admite nuevas asignaciones, retiros ni resultados.

Estado inicial	**Operación**	**Estado final**	**Condiciones y efectos**
No existe	Crear jornada	EN_PREPARACION	La cuadrilla existe y no tiene otra jornada sin cerrar. La jornada

			comienza sin órdenes ni resultados.
EN_PREPARACION	Asignar una orden	EN_PREPARACION	La orden está pendiente, es compatible y cabe dentro de la capacidad disponible.
EN_PREPARACION	Retirar una orden	EN_PREPARACION	La orden pertenece a la jornada; se retira y vuelve a pendiente.
EN_PREPARACION	Iniciar jornada	EN_ATENCION	Debe contener al menos una orden. Se inicia una visita para cada orden y se generan los cargos adicionales que correspondan.
EN_ATENCION	Registrar un resultado cuando todavía faltan otros	EN_ATENCION	La orden pertenece a la jornada y no tiene un resultado registrado en ella. Se actualiza la orden según el resultado y sus visitas disponibles.
EN_ATENCION	Registrar el último resultado pendiente	CERRADA	Todas las órdenes tienen resultado registrado. La cuadrilla queda disponible para una nueva jornada.

Las operaciones que no correspondan al estado actual serán rechazadas sin modificar los datos ni tampoco los ingresos.

Casos de rechazo explícito de operación (sin alteración de estado ni ingresos):

- Asignar una orden que no esté pendiente (ya asignada, en atención, completada o cancelada).
- Retirar una orden que no esté asignada, o cuya jornada ya inició.
- Registrar resultado a una orden que no esté en atención.

- Las órdenes completadas o canceladas no pueden asignarse nuevamente, iniciar otra visita ni recibir nuevos resultados. Sus datos se conservan y pueden consultarse e incluirse en los reportes.

3.3 Tabla que conecta ambas máquinas de estados

Operación del usuario	**Efecto en la orden**	**Efecto en la jornada**	**Efecto en ingresos**
Registrar orden	Nace pendiente	n/a	+ monto completo
Cotizar	Nada	Nada	Nada
Crear jornada	Nada	Nace en preparación	Nada
Asignar orden	Pendiente → asignada	Se agrega a su lista	Nada
Retirar orden	Asignada → pendiente	Se quita de su lista	Nada
Iniciar jornada	Asignada → en atención; visitas + 1	En preparación → en atención	+ Q40 por cada orden con visita ≥ 2
Registrar resultado	En atención → completada, pendiente o cancelada	Cierra si era la última	Nada
Operación rechazada	Sin cambios	Sin cambios	Nada

4. Clases y responsabilidades

Cada orden conserva lo que ha generado y los reportes de ingreso se calculan sumando sobre las órdenes registradas, de modo que existe un único dato y no puede contradecirse.

4.1 Listado general

Clase	**Tipo**	**Propósito**
Zona	enum	Ubicación del servicio: METROPOLITANA, PERIFERICA
EstadoOrden	enum	PENDIENTE, ASIGNADA, EN_ATENCION, COMPLETADA, CANCELADA
EstadoJornada	enum	EN_PREPARACION, EN_ATENCION, CERRADA

ResultadoVisita	enum	COMPLETADA, FALLIDA: resultado que el personal registra
Tarifas	clase final	Única fuente de la tarifa base por zona (100 / 175)
OrdenServicio	abstracta	Datos y comportamiento comunes; declara lo que cada categoría debe decidir
Diagnostico	concreta	Orden de diagnóstico
Instalacion	concreta	Orden de instalación
InstalacionElectricaCertificada	concreta, hereda de Instalacion	Instalación con certificado y garantía
MantenimientoAireAcondicionado	concreta	Mantenimiento de A/C
Cuadrilla	concreta	Capacidad y certificación de una cuadrilla
Jornada	concreta	Agrupa órdenes de una cuadrilla, controla capacidad y compatibilidad, y su ciclo de vida
SistemaTecniHogar	concreta	Coordinadora: colecciones, unicidad de códigos, operaciones y reportes
Main	concreta	Contiene main, el menú y la lectura segura de entradas

4.2 Detalle de cada clase

Tarifas

Elemento	Tipo	Visib.	Propósito
BASE_METROPOLITANA	double constante (private static final)	-	100.0
BASE_PERIFERICA	double constante (private static final)	-	175.0

Tarifas()	constructor	-	Privado: la clase no se instancia
baseServicio(Zona zona)	static, devuelve double	+	Tarifa base de la zona; punto único de cambio

OrdenServicio (abstracta)

Atributos

Nombre	Tipo	Visib.	Propósito
codigo	String (final)	-	Identificador único (la unicidad la verifica el sistema)
cliente	String (final)	-	Nombre del cliente
direccion	String (final)	-	Dirección del servicio
horasEstimadas	double (final)	-	Horas de trabajo (admite decimales). Usadas para capacidad
zona	Zona (final)	-	Metropolitana o periférica
estado	EstadoOrden	-	Estado actual. Solo cambia mediante los métodos de transición
visitas	int	-	Visitas realizadas (se cuentan al iniciar la jornada)
montoRegistro	double	-	Monto cobrado al registrar. Vale 0 mientras no se registre
cargosVisitasAdicionales	double	-	Suma de los Q40 acumulados
CARGO_VISITA_ADICIONAL	double (private static final)	-	40.0, igual para todas las categorías
registroCobrado	boolean	-	Comienza en false e impide cobrar el registro más de una vez

Métodos

Método	Parámetros	Resultado	Visib.	Propósito
protected OrdenServicio(String codigo, String cliente, String direccion, double horasEstimadas, Zona zona)	los de la firma	n/a	#	Valida lo común e inicia estado y contadores
calcularCosto()	ninguno	double	+ abstracto	Costo según la categoría
getDiasAtencion()	ninguno	int	+ abstracto	Días hábiles estimados
getMaximoVisitas()	ninguno	int	+ abstracto	Política de visitas
getCategoria()	ninguno	String	+ abstracto	Nombre para reportes
esCompatibleCon(Cuadrilla c)	cuadrilla	boolean	+	Por defecto true. El A/C la sobrescribe
cobrarRegistro()	ninguno	void	+	Guarda montoRegistro = calcularCosto(). Lo llama el sistema al registrar. Falla si ya se cobró
asignar()	ninguno	void	+	PENDIENTE → ASIGNADA. Falla si no está pendiente
retirar()	ninguno	void	+	ASIGNADA → PENDIENTE. Falla si no está asignada

iniciarVisita()	ninguno	void	+	ASIGNADA → EN_ATENCION, visitas++; si visitas ≥ 2, suma Q40 a cargosVisitasAdicionales
registrarResultado(ResultadoVisita r)	resultado	void	+	Solo desde EN_ATENCION. Completada → COMPLETADA. Fallida → CANCELADA si visitas ≥ getMaximoVisitas(), si no PENDIENTE
getIngresoTotal()	ninguno	double	+	montoRegistro + cargosVisitasAdicionales (para el reporte por categoría)
getCodigo()	ninguno	String	+	Consultar el código
getCliente()	ninguno	String	+	Consultar el nombre del cliente
getDireccion()	ninguno	String	+	Consultar la dirección
getHorasEstimadas()	ninguno	double	+	Consultar las horas utilizadas para controlar la capacidad
getZona()	ninguno	Zona	+	Consultar la zona del servicio
getEstado()	ninguno	EstadoOrden	+	Consultar el estado actual
getVisitas()	ninguno	int	+	Consultar las visitas realizadas
getMontoRegistro()	ninguno	double	+	Consultar el ingreso generado por el registro

getCargosVisitasAdicionales()	ninguno	double	+	Consultar los cargos acumulados por visitas adicionales
toString()	ninguno	String	+	Sobrescribe el método de Object. Devuelve un resumen para consultas y no modifica datos

Diagnostico (hereda de OrdenServicio)

Elemento	**Tipo / resultado**	**Visib.**	**Propósito**
HORAS_MAXIMAS	double (private static final), 1.5	-	Límite propio
TARIFA_METROPOLITANA, TARIFA_PERIFERICA	double (private static final), 150 y 225	-	Tarifa fija propia
public Diagnostico(String codigo, String cliente, String direccion, double horasEstimadas, Zona zona)	n/a	+	Llama a super(...) y valida horas ≤ 1.5
calcularCosto()	double	+	Sobrescribe: tarifa fija por zona
getDiasAtencion()	int	+	Sobrescribe: 1 metropolitana, 2 periférica
getMaximoVisitas()	int	+	Sobrescribe: 3
getCategoria()	String	+	Sobrescribe: “Diagnóstico”

Instalacion (hereda de OrdenServicio)

Elemento	Tipo / resultado	Visib.	Propósito
puntos	int (final)	-	Puntos a instalar (> 0)
TARIFA_POR_HORA, HORAS_POR_PUNTO	double (private static final), 90 y 0.75	-	Reglas propias
public Instalacion(String codigo, String cliente, String direccion, double horasEstimadas, Zona zona, int puntos)	n/a	+	super(...) y valida puntos > 0
getPuntos()	int	+	Getter
getHorasPorPuntos()	double	+	puntos × 0.75
getHorasFacturables()	int	+	ceil(max(horasEstimadas, horasPorPuntos))
calcularCosto()	double	+	Sobrescribe: Tarifas.baseServicio(getZona()) + 90 × horasFacturables
getDiasAtencion()	int	+	Sobrescribe: 3 / 5
getMaximoVisitas()	int	+	Sobrescribe: 3
getCategoria()	String	+	Sobrescribe: “Instalación”

InstalacionElectricaCertificada (hereda de Instalacion)

Elemento	Tipo / resultado	Visib.	Propósito
valorMateriales	double (final)	-	Valor declarado de los materiales

CARGO_CERTIFICADO, PORCENTAJE_GARANTIA	double (private static final), 120 y 0.05	-	Reglas propias
public InstalacionElectricaCertificada(String codigo, String cliente, String direccion, double horasEstimadas, Zona zona, int puntos, double valorMateriales)	n/a	+	Llama a super(...) y valida que valorMateriales sea finito y estrictamente mayor que cero.
getGarantia()	double	+	0.05 × valorMateriales
calcularCosto()	double	+	Sobrescribe: super.calcularCosto() + 120 + getGarantia()
getDiasAtencion()	int	+	Sobrescribe: super.getDiasAtencion() + 2
getMaximoVisitas()	int	+	Sobrescribe: 2
getCategoria()	String	+	Sobrescribe: “Instalación eléctrica certificada”
getValorMateriales()	double	+	Consultar el valor declarado de los materiales.

MantenimientoAireAcondicionado (hereda de OrdenServicio)

Elemento	Tipo / resultado	Visib.	Propósito
capacidadBtu	int (final)	-	Capacidad del equipo
BTU_MINIMO, BTU_MAXIMO	int (private static final),	-	Límites de BTU

	9000 y 60000		
TARIFA_POR_HORA, CARGO_GAS	double (private static final), 110 y 80	-	Reglas propias
public MantenimientoAireAcondicionado(String codigo, String cliente, String direccion, double horasEstimadas, Zona zona, int capacidadBtu)	n/a	+	super(...), valida zona metropolitana y BTU en rango
calcularCosto()	double	+	Sobrescribe: Tarifas.baseServicio(getZona()) + 110 × horas + 80
getDiasAtencion()	int	+	Sobrescribe: 2
getMaximoVisitas()	int	+	Sobrescribe: 1
esCompatibleCon(Cuadrilla c)	boolean	+	Sobrescribe: c.isCertificadaRefrigeracion()
getCategoria()	String	+	Sobrescribe: “Mantenimiento de aire acondicionado”
getCapacidadBtu()	int	+	Consultar la capacidad del equipo.

Cuadrilla

Elemento	**Tipo / resultado**	**Visib.**	**Propósito**
codigo	String (final)	-	Identificador único
capacidadHoras	double (final)	-	Máximo de horas estimadas por jornada (> 0)
certificadaRefrigeracion	boolean (final)	-	Habilita mantenimientos de A/C

public Cuadrilla(String codigo, double capacidadHoras, boolean certificadaRefrigeracion)	n/a	+	Valida código no vacío y capacidad > 0
getCodigo()	String	+	Consultar el código de la cuadrilla
getCapacidadHoras()	double	+	Consultar su capacidad máxima por jornada
isCertificadaRefrigeracion()	boolean	+	Consultar si está certificada en refrigeración
toString()	String	+	Resumen de la cuadrilla para consultas; sobrescribe Object.toString(), sin parámetros

Jornada

Atributos

Nombre	Tipo	Visib.	Propósito
numero	int (final)	-	Identificador asignado por el sistema
cuadrilla	Cuadrilla (final)	-	Cuadrilla que la atiende
ordenes	List<OrdenServicio> (final)	-	Órdenes asignadas
resultados	Map<OrdenServicio, ResultadoVisita> (final)	-	Resultados ya registrados en esta jornada
estado	EstadoJornada	-	Estado actual

Métodos

Método	Parámetros	Resultado	Visib.	Propósito

public Jornada(int numero, Cuadrilla cuadrilla)	los de la firma	n/a	+	Nace EN_PREPARACION con listas vacías
asignarOrden(OrdenServicio o)	orden	void	+	Valida: en preparación, o.esCompatibleCon(cuadrilla), horasAsignadas + o.getHorasEstimadas() ≤ capacidad. Luego o.asignar() y la agrega
retirarOrden(OrdenServicio o)	orden	void	+	Solo en preparación y si le pertenece. o.retirar() y la quita
iniciar()	ninguno	void	+	Valida en preparación y no vacía. Llama iniciarVisita() a cada orden y pasa a EN_ATENCION
registrarResultado(OrdenServicio o, ResultadoVisita r)	orden, resultado	void	+	Valida en atención, pertenece y sin resultado previo. Llama o.registrarResultado(r). Cierra si ya todas tienen resultado
getHorasAsignadas()	ninguno	double	+	Suma de horas estimadas
getCapacidadDisponible()	ninguno	double	+	Capacidad − horas asignadas
estaCerrada()	ninguno	boolean	+	estado == CERRADA
getNumero()	ninguno	int	+	Consultar el identificador de la jornada
getCuadrilla()	ninguno	Cuadrilla	+	Consultar la cuadrilla responsable

getOrdenes()	ninguno	List<OrdenServicio>	+	Consultar las órdenes mediante una lista de solo lectura
getEstado()	ninguno	EstadoJornada	+	Consultar el estado de la jornada
toString()	ninguno	String	+	Resumen de la jornada para consultas; sobrescribe Object.toString()
exigirEnPreparacion()	Ninguno	void	-	Rechazar operaciones que requieren una jornada en preparación.
exigirPertenencia(OrdenServicio orden)	Ninguno	void	-	Comprobar que la orden pertenece a la jornada.
sumarHorasAsignadas()	Ninguno	BigDecimal	-	Sumar las horas estimadas para controlar la capacidad.

SistemaTecniHogar

Atributos (todos -)

Nombre	Tipo	Propósito
ordenes	Map<String, OrdenServicio>	Órdenes registradas por código
cuadrillas	Map<String, Cuadrilla>	Cuadrillas por código
jornadas	List<Jornada>	Todas las jornadas
siguienteNumeroJornada	int	Contador para numerar jornadas
normalizarCodigo(String codigo)	String	Validar el código y quitar espacios al inicio y al final.

Métodos

Método	Resultado	Propósito

public SistemaTecniHogar()	n/a	Crea el sistema con colecciones vacías y el contador de jornadas en 1
cotizar(OrdenServicio o)	String	Costo y días. No modifica nada
registrarOrden(OrdenServicio o)	void	Verifica código único, o.cobrarRegistro() y guarda
registrarCuadrilla(Cuadrilla c)	void	Verifica código único y guarda
crearJornada(String codCuadrilla)	Jornada	Verifica que exista y no tenga otra jornada sin cerrar
asignarOrden(int numJornada, String codOrden)	void	Busca ambos y delega en Jornada
retirarOrden(int numJornada, String codOrden)	void	Delega
iniciarJornada(int numJornada)	void	Delega
registrarResultado(int numJornada, String codOrden, ResultadoVisita r)	void	Delega
buscarOrden(String codigo)	OrdenServicio	Buscar una orden por su código; indicar un error si no existe
buscarCuadrilla(String codigo)	Cuadrilla	Buscar una cuadrilla por su código; indicar un error si no existe
buscarJornada(int numero)	Jornada	Buscar una jornada por su número; indicar un error si no existe
listarOrdenes()	List<OrdenServicio>	Devolver una lista de solo lectura con las órdenes registradas
listarCuadrillas()	List<Cuadrilla>	Devolver una lista de solo lectura con las cuadrillas registradas

contarPorCategoriaYEstado()	Map<String, Map<EstadoOrden, Integer>>	Reporte 1
getIngresosPorRegistro(), getIngresosPorVisitas(), getIngresoTotal()	double	Reporte 2 (suma sobre las órdenes)
getIngresosPorCategoria()	Map<String, Double>	Reporte 2
getJornadasNoCerradas()	List<Jornada>	Reporte 3
cargarDatosIniciales()	void	Incorporar ocho órdenes y tres cuadrillas sin cobrar registros; exige que el sistema esté vacío.

Main

Firma	**Propósito**
public static void main(String[] args)	Preparar el sistema y la entrada de datos, iniciar la carga inicial y mantener el ciclo del menú hasta que el usuario elija salir.
private static void mostrarMenu()	Mostrar las opciones disponibles.
private static void ejecutarOpcion(int opcion, SistemaTecniHogar sistema, Scanner entrada)	Capturar los datos necesarios para la opción y solicitar la operación correspondiente al sistema.
private static int leerEntero(Scanner entrada, String mensaje)	Solicitar un entero y repetir la lectura cuando el formato sea incorrecto.
private static double leerDecimal(Scanner entrada, String mensaje)	Solicitar un número decimal y repetir la lectura cuando el formato sea incorrecto.
private static String leerTexto(Scanner entrada, String mensaje)	Solicitar una entrada de texto.
private static void cargarDatosIniciales(SistemaTecniHogar sistema)	Coordinar la incorporación de las órdenes y cuadrillas iniciales, sin generar ingresos por registro.
Main()	Impedir la creación de instancias de esta clase de entrada.

Relaciones previstas para el diagrama

Relación	Multiplicidad
SistemaTecniHogar → OrdenServicio	1 a 0..*
SistemaTecniHogar → Cuadrilla	1 a 0..*
SistemaTecniHogar → Jornada	1 a 0..*
Jornada → Cuadrilla	0..* a 1 (una cuadrilla tiene máximo una jornada sin cerrar)
Jornada → OrdenServicio	1 a 0..* (una orden está en una sola jornada activa a la vez)
Instalacion / MantenimientoAireAcondicionado → Tarifas	dependencia (uso estático)
Main → SistemaTecniHogar	1 a 1

4.3 Flujo del programa por medio de clases y enum

Los objetos reciben sus datos iniciales mediante sus constructores. La captura de esos datos corresponde a Main, mientras que la validación de las reglas del dominio corresponde a las clases responsables de representarlas.

El constructor de OrdenServicio recibe el código, el cliente, la dirección, las horas estimadas y la zona. Valida los datos comunes e inicializa el estado como PENDIENTE, las visitas en cero, el monto de registro en cero y los cargos por visitas adicionales en cero. Crear el objeto no genera un cobro; este se realiza cuando el sistema acepta su registro.

Los constructores de Diagnostico, Instalacion y MantenimientoAireAcondicionado llaman al constructor de OrdenServicio mediante super(...). Después validan e inicializan los datos particulares de su categoría.

El constructor de InstalacionElectricaCertificada llama al de Instalacion, que a su vez llama al de OrdenServicio. Así, OrdenServicio inicializa los datos comunes, Instalacion valida y conserva los puntos, e InstalacionElectricaCertificada valida y conserva el valor de los materiales. La validación de los datos compartidos se reutiliza mediante esta cadena de constructores.

Cuadrilla recibe su código, capacidad y certificación. Su constructor valida el código y la capacidad; SistemaTecniHogar comprueba que el código no esté repetido.

Jornada recibe su número y la cuadrilla responsable. Comienza en estado EN_PREPARACION, con una lista de órdenes y un mapa de resultados vacíos. SistemaTecniHogar asigna su número y comprueba que la cuadrilla pueda recibir una nueva jornada.

SistemaTecniHogar comienza con las colecciones de órdenes, cuadrillas y jornadas vacías, y con el contador de jornadas en uno. La incorporación posterior de los datos iniciales respeta la excepción de precarga sin ingresos.

Tarifas tiene un constructor privado porque no necesita instancias. Sus valores compartidos se consultan mediante su método estático.

Las comprobaciones de unicidad pertenecen a SistemaTecniHogar porque requieren conocer los objetos ya registrados. Los constructores validan la información del objeto que están creando.

Sección 5

InstalacionElectricaCertificada se relaciona con Instalacion mediante herencia porque representa una especialización de una instalación. Conserva los datos comunes de la orden, la cantidad de puntos y las reglas para determinar las horas facturables, pero incorpora el valor de los materiales y comportamientos específicos.

La clase reutiliza el cálculo de Instalacion mediante super.calcularCosto() y añade Q120 por el certificado y el 5 % del valor declarado de los materiales. También reutiliza el tiempo de una instalación mediante super.getDiasAtencion() y agrega dos días. Sobrescribe getMaximoVisitas() para permitir dos visitas y getCategoria() para identificarse como una categoría diferente en consultas y reportes.

Esta relación evita duplicar las reglas de horas por puntos, redondeo de horas facturables y costo básico de instalación. También permite tratar una instalación eléctrica certificada como una OrdenServicio y ejecutar su comportamiento particular mediante polimorfismo. El máximo de tres visitas de Instalacion representa su comportamiento normal, que la especialización puede sobrescribir; no es una regla que deba imponerse a todas sus subclases.

Una alternativa sería hacer que Instalacion e InstalacionElectricaCertificada heredaran directamente de OrdenServicio. Esa organización reduciría la dependencia directa entre ambas clases, pero obligaría a repetir las reglas comunes de instalación o a extraerlas en un componente reutilizable.

Se elige la herencia desde Instalacion porque el enunciado define la instalación eléctrica certificada como una instalación con requisitos adicionales. La desventaja es que un cambio en el comportamiento de Instalacion puede afectar a su subclase y debe revisarse para comprobar que siga siendo adecuado. Si ambas categorías dejaran de compartir esas reglas en el futuro, sería conveniente reconsiderar la relación.

6. Impacto de agregar una nueva categoría

Para incorporar la categoría hipotética de instalación de paneles solares se creará InstalacionPanelesSolares como subclase de OrdenServicio. Esta conservará la cantidad de paneles e implementará sus reglas de costo, días de atención, máximo de visitas, categoría y compatibilidad.

En este ejemplo, la compatibilidad exige certificación en electricidad solar. Por tanto, InstalacionPanelesSolares debe sobrescribir esCompatibleCon(Cuadrilla), y Cuadrilla debe poder representar y permitir consultar esa certificación.

6.1 Caso de ejemplo

Aspecto	Regla hipotética

Datos propios	Cantidad de paneles (entero positivo)
Costo	Tarifa base de zona + Q500 por panel
Días de atención	6 metropolitana, 9 periférica
Visitas máximas	2
Cuadrilla	Solo cuadrillas certificadas en electricidad solar
Validación	Mínimo 1 panel; se acepta en ambas zonas

6.2 Clases que se crean

InstalacionPanelesSolares

Elemento	**Qué implementa**
Atributo cantidadPaneles	Dato propio, private final
Constructor	Llama a super(...) de OrdenServicio y valida paneles > 0
calcularCosto()	Tarifas.baseServicio(getZona()) + 500 × cantidadPaneles
getDiasAtencion()	6 / 9 según zona
getMaximoVisitas()	2
getCategoria()	“Instalación de paneles solares”
esCompatibleCon(Cuadrilla)	Se sobrescribe para exigir certificación en electricidad solar.

6.3 Modificación de Clases

Clase o grupo	**Cambio necesario**	**Justificación**
InstalacionPanelesSolares	Crear	Conserva sus datos particulares e implementa las reglas de la nueva categoría.
Cuadrilla	Modificar	Actualmente solo representa la certificación en refrigeración; debe incorporar la certificación requerida por el ejemplo solar.
Main	Modificar	Debe ofrecer la nueva categoría, capturar la cantidad de paneles y construir la orden. También debe

		permitir capturar la nueva certificación al registrar cuadrillas.
OrdenServicio	No requiere cambios para este ejemplo	Sus métodos permiten expresar las reglas propuestas de costo, tiempo, compatibilidad y máximo de visitas.
Categorías existentes	No requieren cambios	Mantienen sus datos y comportamientos actuales.
Jornada	No requiere cambios para este ejemplo	Continúa controlando las horas estimadas y consulta la compatibilidad mediante esCompatibleCon(Cuadrilla).
SistemaTecniHogar	No requiere cambios para este ejemplo	Sigue registrando y consultando objetos de OrdenServicio y coordinando las mismas operaciones.
Reportes	No requieren cambios	Agrupan por getCategoria() y consultan estados, montos y datos de las jornadas.
Tarifas	No requiere cambios para este ejemplo	La nueva categoría utiliza las mismas tarifas base existentes.
Zona, EstadoOrden, EstadoJornada y ResultadoVisita	No requieren cambios	El ejemplo no introduce nuevas zonas, estados ni resultados.

6.4 Reportes y jornadas, con evidencia

Pregunta	**Respuesta**	**Evidencia**
¿Cambia el reporte de órdenes por categoría y estado?	No	Agrupa por getCategoria(); la nueva categoría aparece como una clave más
¿Cambia el reporte de ingresos?	No	Suma montoRegistro y cargosVisitasAdicionales de cada orden; el ingreso por categoría agrupa con getCategoria()
¿Cambia el reporte de jornadas no cerradas?	No	Usa estado de la jornada, cuadrilla, horas estimadas y capacidad
¿Cambia Jornada?	No	Su validación de compatibilidad es una pregunta polimórfica: orden.esCompatibleCon(cuadrilla)

¿Cambia la política de visitas o el cargo de Q40?	No	OrdenServicio consulta getMaximoVisitas() de la nueva clase

Agregar una categoría requiere crear una subclase de OrdenServicio que implemente sus reglas de costo, tiempo, visitas, nombre y compatibilidad, y modificar el menú para capturar sus datos. Las jornadas, el sistema coordinador y los reportes no cambian, porque solo se comunican con la clase base mediante métodos polimórficos. El diseño tiene límites conocidos: si la categoría exige una certificación distinta de refrigeración, Cuadrilla se tiene que modificar, si altera el consumo de capacidad o el cargo por visita adicional, esas reglas deben convertirse en métodos sobrescribibles de la base.

7. **Pruebas realizadas**

7.1. Reglas de cobro, tiempos y cotizaciones sin ingresos

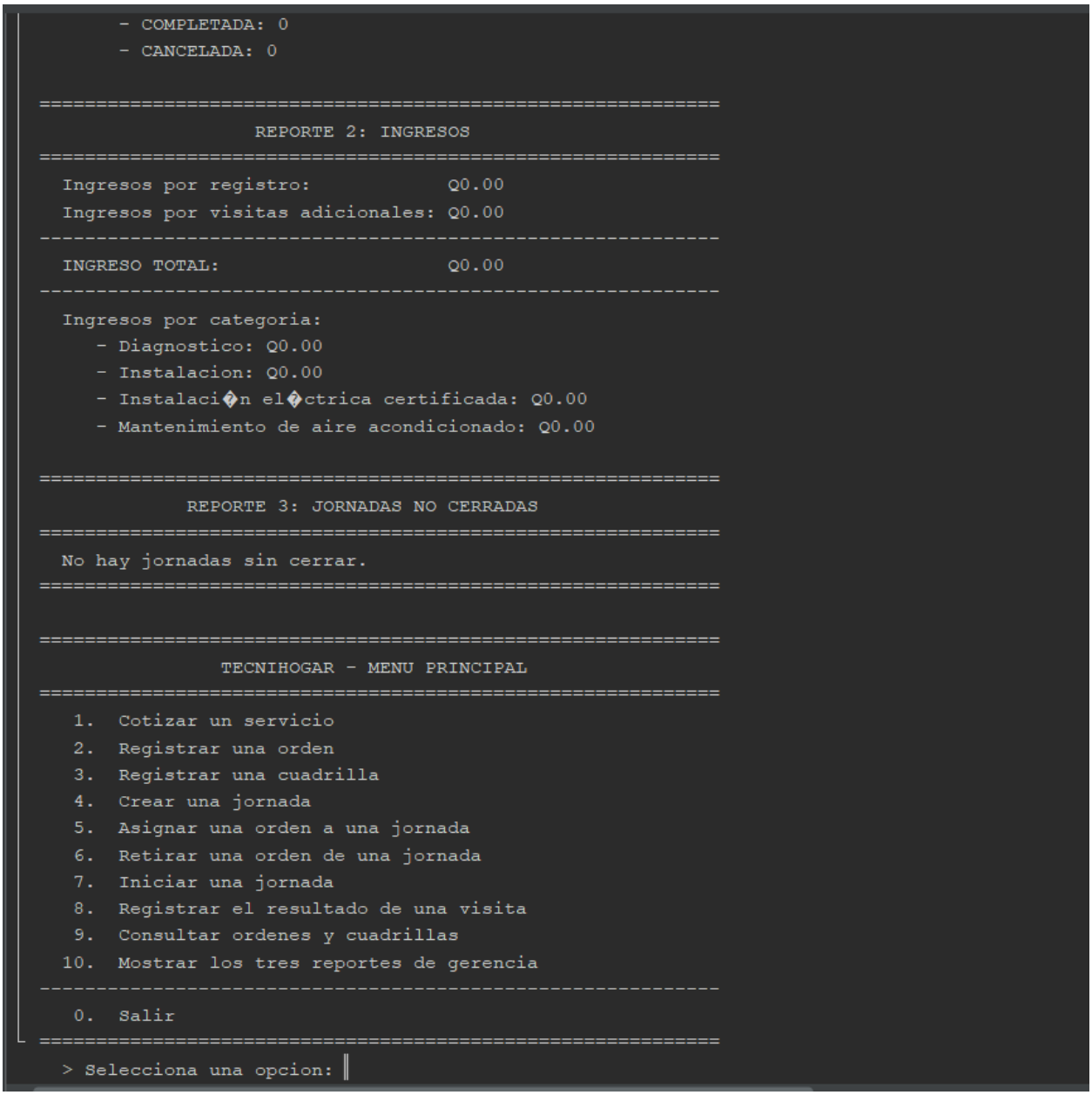
Caso	**Datos de entrada**	**Resultado esperado**	**Resultado obtenido**
Diagnóstico metropolitano	1 hora; zona metropolitana	Q150.00; 1 día	Coincide con el resultado esperado
Diagnóstico periférico	1 hora; zona periférica	Q225.00; 2 días	Coincide con el resultado esperado
Instalación con fracción de hora	2.1 horas; 1 punto; zona periférica	Se cobran 3 horas: Q445.00; 5 días	Coincide con el resultado esperado
Instalación certificada con horas por puntos y garantía	2.5 horas; 5 puntos; materiales Q2000; zona metropolitana	3.75 horas por puntos, redondeadas a 4. Q460 + Q120 + Q100 = **Q680.00**; 5 días	Coincide con el resultado esperado
Mantenimiento de aire acondicionado	2 horas; 9000 BTU; zona metropolitana	Q100 + Q220 + Q80 = **Q400.00**; 2 días	Coincide con el resultado esperado

Comprobación	**Antes**	**Esperado después**	**Obtenido después**
Ingresos tras las cinco cotizaciones	Q0.00	Q0.00	Coincide con el resultado esperado

```
t - Run (TecniHogar)
 ============================================================
   > Selecciona una opcion: .10
   [!] Ingresa un numero entero valido, por ejemplo 3.
   > Selecciona una opcion: 10

 ============================================================
        REPORTE 1: ORDENES POR CATEGORIA Y ESTADO
 ============================================================
   * Diagnostico:
        - PENDIENTE: 2
        - ASIGNADA: 0
        - EN_ATENCION: 0
        - COMPLETADA: 0
        - CANCELADA: 0
   * Instalacion:
        - PENDIENTE: 2
        - ASIGNADA: 0
        - EN_ATENCION: 0
        - COMPLETADA: 0
        - CANCELADA: 0
   * Instalaci□n el□ctrica certificada:
        - PENDIENTE: 2
        - ASIGNADA: 0
        - EN_ATENCION: 0
        - COMPLETADA: 0
        - CANCELADA: 0
   * Mantenimiento de aire acondicionado:
        - PENDIENTE: 2
        - ASIGNADA: 0
        - EN_ATENCION: 0
        - COMPLETADA: 0
        - CANCELADA: 0

 ============================================================
                   REPORTE 2: INGRESOS
 ============================================================
   Ingresos por registro:            Q0.00
   Ingresos por visitas adicionales: Q0.00
 ------------------------------------------------------------
```



7.2. Capacidad, certificación y retiro de órdenes

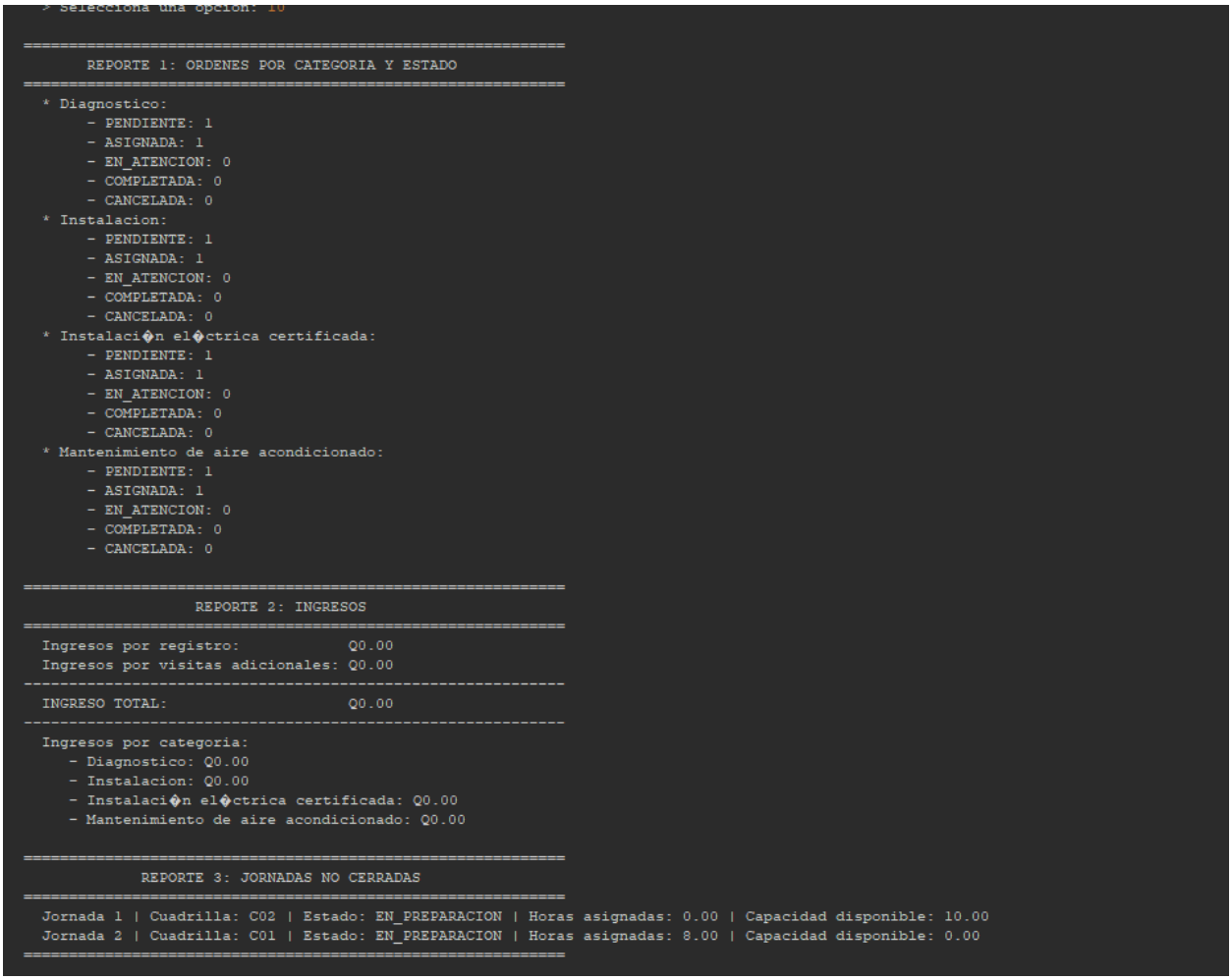
```
 2.  Registrar una orden
 3.  Registrar una cuadrilla
 4.  Crear una jornada
 5.  Asignar una orden a una jornada
 6.  Retirar una orden de una jornada
 7.  Iniciar una jornada
 8.  Registrar el resultado de una visita
 9.  Consultar ordenes y cuadrillas
10.  Mostrar los tres reportes de gerencia
------------------------------------------------------------
 0.  Salir
============================================================
> Selecciona una opcion: 5

----------------- ASIGNAR ORDEN --------------------------
> Numero de jornada: 2
> Codigo de la orden: D02

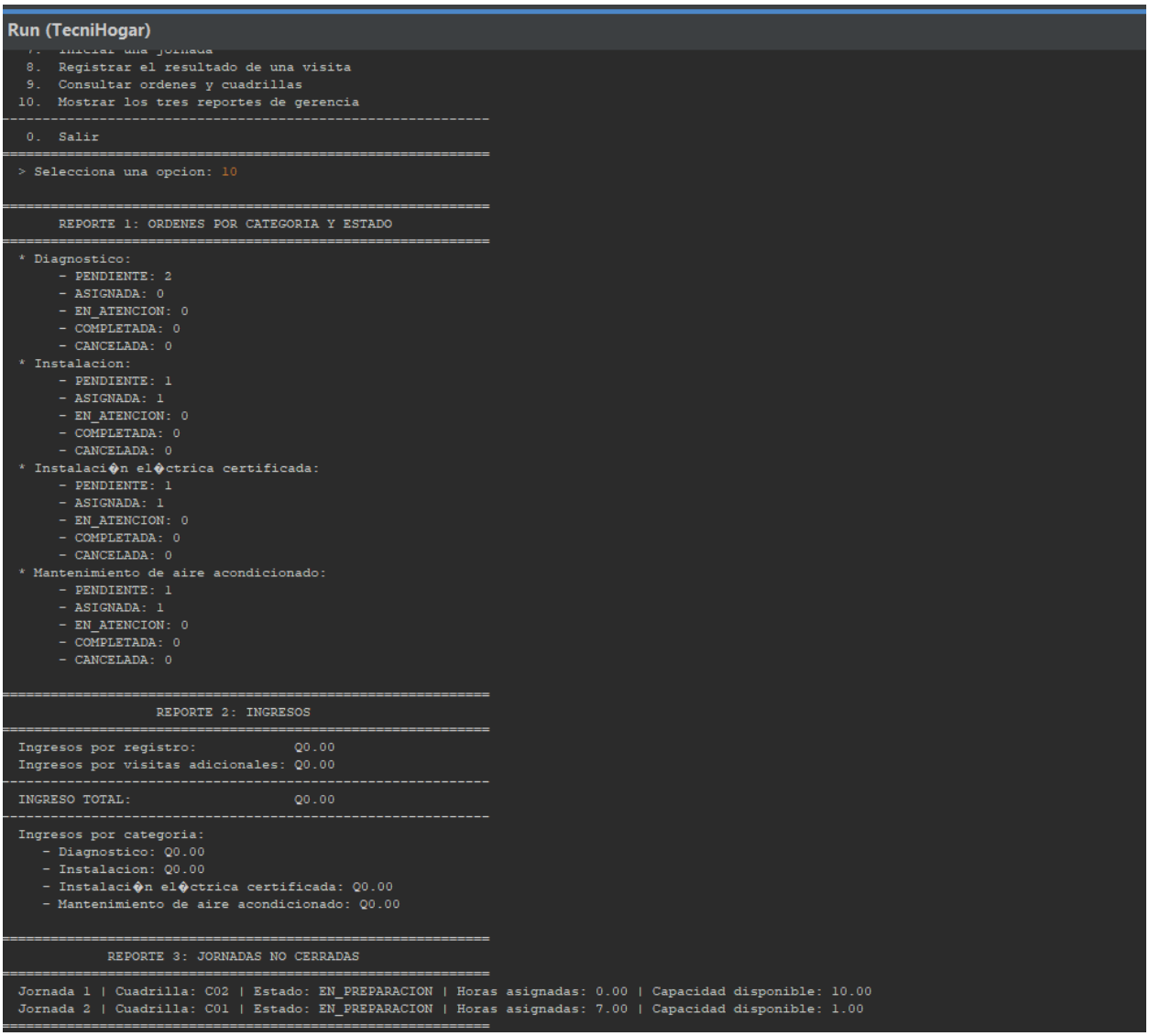
[ERROR] La orden requiere 1.50 horas, pero la jornada solo tiene 0.00 horas disponibles.

============================================================
              TECNIHOGAR - MENU PRINCIPAL
============================================================
 1.  Cotizar un servicio
 2.  Registrar una orden
 3.  Registrar una cuadrilla
 4.  Crear una jornada
 5.  Asignar una orden a una jornada
 6.  Retirar una orden de una jornada
 7.  Iniciar una jornada
 8.  Registrar el resultado de una visita
 9.  Consultar ordenes y cuadrillas
10.  Mostrar los tres reportes de gerencia
------------------------------------------------------------
 0.  Salir
============================================================
> Selecciona una opcion:
```

Rechazo por falta de incompatibilidad



Despues de Retirar el diagnóstico



Caso	Datos de entrada	Resultado esperado	Resultado obtenido
Restricción de certificación	Asignar A01 a la jornada 1 de C02, sin certificación en refrigeración.	Asignación rechazada por incompatibilidad; sin cambios en órdenes ni ingresos.	Coincide con el resultado esperado
Capacidad exacta	Asignar D01, I01, E01 y A01 a la jornada 2	Asignaciones aceptadas: 8 horas estimadas y 0 horas disponibles.	Coincide con el resultado esperado

	de C01, con capacidad de 8 horas.		
Exceso de capacidad	Agregar D02, de 1.5 horas, cuando quedan 0 horas disponibles.	Asignación rechazada; se mantienen las 8 horas asignadas.	Coincide con el resultado esperado
Ingresos después de operaciones rechazadas	Consultar ingresos después de los intentos inválidos.	Ingresos por registro y visitas adicionales iguales a Q0.00.	El reporte compartido muestra ambos ingresos y el total en Q0.00.
Retiro de una orden	Retirar D01 de la jornada 2 en preparación.	D01 vuelve a PENDIENTE; quedan 7 horas asignadas y 1 disponible; ingresos Q0.00.	El reporte compartido muestra 7 horas asignadas, 1 disponible, los dos diagnósticos pendientes e ingresos Q0.00.
Resultado antes de iniciar la jornada	Intentar registrar visitas fallidas en la jornada 2, todavía EN_PREPARACION.	Operaciones rechazadas sin cambiar estados ni ingresos.	Apareció “La jornada debe estar en atención”. El reporte posterior mantuvo la jornada en preparación y los ingresos en Q0.00.

Caso y datos de entrada	**Resultado esperado**	**Resultado obtenido**
Asignar A01 a jornada 1, cuadrilla C02 sin certificación	Rechazo por incompatibilidad	Coincide con el resultado esperado
Asignar D01, I01, E01 y A01 a jornada 2	Se aceptan exactamente 8 horas estimadas	Coincide con el resultado esperado
Agregar D02 cuando quedan 0 horas disponibles	Rechazo por exceso de capacidad	Coincide con el resultado esperado
Consultar ingresos después de ambos rechazos	Permanecen en Q0.00	Coincide con el resultado esperado

Retirar D01 de jornada 2	PENDIENTE; quedan 7 horas asignadas y 1 disponible; ingresos permanecen en Q0.00	Coincide con el resultado esperado

7.3. Límites de visitas y cargos adicionales

Orden y categoría	Datos de prueba	Estado final esperado	Visitas esperadas	Cargos adicionales esperados	Resultado obtenido
D01 — Diagnóstico	Tres visitas iniciadas y registradas como fallidas en jornadas distintas.	CANCELADA	3	Q80.00	Coincide con el resultado esperado
I01 — Instalación	Tres visitas iniciadas y registradas como fallidas en jornadas distintas.	CANCELADA	3	Q80.00	Coincide con el resultado esperado
E01 — Instalación eléctrica certificada	Dos visitas iniciadas y registradas como fallidas en jornadas distintas.	CANCELADA	2	Q40.00	Coincide con el resultado esperado
A01 — Mantenimiento de aire acondicionado	Una visita iniciada y registrada como fallida.	CANCELADA	1	Q0.00	Coincide con el resultado esperado

Momento de consulta	Ingresos por registro esperados	Ingresos por visitas adicionales esperados	Total esperado	Resultado obtenido

Después de iniciar la primera visita de las cuatro órdenes.	Q0.00	Q0.00	Q0.00	Coincide con el resultado esperado
Después de iniciar la segunda visita de D01, I01 y E01, antes de registrar resultados.	Q0.00	Q120.00	Q120.00	Coincide con el resultado esperado
Después de iniciar la tercera visita de D01 e I01.	Q0.00	Q200.00	Q200.00	Coincide con el resultado esperado

Diagrama UML:

